

Project 4 - Documentation

Name: Vishal Chatterjee

AndrewID: vchatter

Overview:

This project is a distributed football analytics system combining an Android app, a Java servlet backend, MongoDB logging, and a JSP-based dashboard. The Android app allows users to search for top scorers, upcoming matches, and team details across major football competitions. These requests are sent to a servlet deployed on Tomcat, which fetches data from the Football-Data.org API and logs each query into MongoDB Atlas with metadata like type, competition, and timestamp.

A web-based dashboard displays aggregated insights from the logs, including the most queried competitions, common query types, recent user activity, and average result counts. The project is fully containerized with Docker for easy deployment and integrates end-to-end data flow across mobile, backend, and database layers.

Android Application:

Key Features

a. **Activities Used:** app consists of three main Activity classes, each serving a specific role:

1. MainActivity.java

This is the entry point of the app and allows users to:

- Enter a football league name (e.g., "Premier League").
- Click on one of the buttons to request:
 - FixturesActivity
 - TeamsActivity
 - TopScorersActivity

Logic:

```
Button fixturesButton = findViewById(R.id.fixturesButton);
fixturesButton.setOnClickListener(v -> {
    String leagueName = inputField.getText().toString().trim();
    if (!leagueName.isEmpty()) {
```

```

        Intent intent = new Intent(MainActivity.this,
FixturesActivity.class);
        intent.putExtra("leagueName", leagueName);
        startActivity(intent);
    } else {
        Toast.makeText(this, "Please enter a league name",
Toast.LENGTH_SHORT).show();
    }
});

```

2. FixturesActivity.java

This screen fetches and displays the next 5 fixtures.

- Uses a TableLayout to dynamically populate matchday, date, home team, and away team.
- Sends HTTP requests using OkHttp.
- Parses JSON returned from your servlet.

Network call (background thread):

```

String url = BASE_URL + "?competitionId=" + competitionId +
"&type=matches";
Request request = new Request.Builder().url(url).build();

client.newCall(request).enqueue(new Callback() {
    @Override
    public void onResponse(@NonNull Call call, @NonNull Response
response) throws IOException {
        JSONArray jsonArray = new
JSONArray(response.body().string());
        runOnUiThread(() -> populateTable(jsonArray));
    }
});

```

populateTable() builds the table:

```

private void populateTable(JSONArray fixtures) {
    for (int i = 0; i < fixtures.length(); i++) {
        JSONObject match = fixtures.getJSONObject(i);
        TableRow row = new TableRow(this);
        row.addView(createTextView(match.getString("homeTeam")));
    }
}

```

```

        row.addView(createTextView(match.getString("awayTeam")));
        tableLayout.addView(row);
    }
}

```

3. TopScorersActivity.java

Displays a leaderboard of top 10 scorers with goals, matches, assists.

- Follows same background-fetch approach using OkHttp.
- UI is also a TableLayout populated from parsed JSON data.

Sample JSON parsing and UI update:

```

String name = player.optString("name", "N/A");
int goals = player.optInt("goals", 0);
TextView nameView = new TextView(this);
nameView.setText(name + " - Goals: " + goals);
tableLayout.addView(nameView);

```

b. HTTP Failure Handling: All OkHttp requests include an onFailure override:

```

@Override
public void onFailure(Call call, IOException e) {
    runOnUiThread(() ->
        Toast.makeText(getApplicationContext(), "Network
error: " + e.getMessage(), Toast.LENGTH_LONG).show());
}

```

c. JSON Parsing Safety: Each field is accessed using optString() or optInt() to avoid exceptions:

```

String position = player.optString("position", "N/A");

```

Workflow / Data Flow

1. User Input (MainActivity)

- Enters league (e.g., "Premier League")
- Clicks a button to go to Fixtures, Teams, or Top Scorers.

2. Send Request

- Activity sends GET request to: `http://<backend-url>/fetchFootballData?competition=Premier%20League&type=matches`

3. Servlet Receives & Responds

- Maps competitionName to competitionId
- Calls Football-Data.org API
- Sends a cleaned JSON response

4. Activity Receives Response

- Parses JSON
- Builds TableLayout with UI rows

5. MongoDB Logs

- Backend logs each request with timestamp, type, result count.

UI Layouts:

Each activity uses:

- A TextView for title.
- A ProgressBar for loading state.
- A TableLayout (within ScrollView) for display.

```
<TableLayout
    android:id="@+id/fixturesTable"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="*"
    android:padding="12dp"/>
```

Servlet:

The FootballDataServlet.java is the central backend endpoint that serves your Android app. It performs 3 main tasks:

1. Receives HTTP GET requests with competition or competitionId and a type (matches, teams, topscorers, or list).
2. Fetches data from the [Football-Data.org API](https://api.football-data.org/).

3. Returns structured JSON to the frontend and logs the event in MongoDB.

Workflow:

Android App → Servlet → Football-Data API → Servlet → JSON Response → Android UI



MongoDB Logs

Request/Response Mapping:

The servlet handles get requests at:

/fetchFootballData?competitionId=2021&type=matches

Or

/fetchFootballData?competition=Premier%20League&type=topscorers

Football-Data API Integration:

Use a static API key to authenticate calls.

Each data type hits a different endpoint:

- Matches: /competitions/{id}/matches
- Teams: /competitions/{id}/teams
- Top Scorers: /competitions/{id}/scorers
- Competitions List: /competitions

Structured JSON Response

Json object returned

```
{  
  "date": "2025-04-21",  
  "homeTeam": "Manchester City",  
  "awayTeam": "Liverpool"  
}
```

Each type handler builds a clean JSONArray, so Android does minimal parsing.

MongoDB Logging

Each successful request is logged to MongoDB Atlas with:

```
private void logToMongoDB(String competitionQuery, String  
dataType, int compId, JSONArray result)
```

Data Stored as:

```
{  
  "competition": "Premier League",  
  "type": "matches",  
  "competitionId": 2021,  
  "resultCount": 5,  
  "timestamp": 1744340780788  
}
```

Error Handling:

Missing parameters:

```
if (compId == -1 || dataType == null || dataType.isEmpty()) {
```

```
        response.setStatus(HttpServletResponse.SC_BAD_REQUEST);

        response.getWriter().write("{\"error\": \"Missing or  
invalid 'competitionId' or 'type' parameter\"}");

        return;
    }
}
```

Football API Failure:

```
catch (IOException | JSONException e) {

    e.printStackTrace();

    return null;

}
```

Servlet Configuration

Using `@WebServlet("/fetchFootballData")` to automatically mapped without needing web.xml.

Dashboard:

The Dashboard module is a web-based analytical interface built using a Java Servlet (Dashboard.java) and a JSP front-end (dashboard.jsp) that provides real-time insights into how users are interacting with your football data platform. Its core purpose is to visualize backend activity—specifically API usage patterns, popular competitions, and user interests—by leveraging log entries stored in a MongoDB Atlas database.

This dashboard is a powerful administrator or analytics view that helps developers, instructors, or stakeholders understand which parts of the football API are being most heavily used, what queries are being run, and what competitions or data types are most popular. The backend servlet uses MongoDB's aggregation pipeline framework to efficiently compute metrics, while the frontend renders them in HTML tables and an interactive Google Pie Chart.

- User interactions on the Android app trigger API calls (e.g., fetching top scorers, matches, teams).
- The FootballDataServlet logs each request to MongoDB.

- The DashboardServlet reads and aggregates these logs, then passes the computed statistics to the dashboard.jsp.
- dashboard.jsp displays tables, textual insights, and a Google Chart.

Code:

<pre>// Mongo Connection MongoClient client = MongoClient.create(MONGO_URI); MongoDatabase db = client.getDatabase("Cluster-task4"); MongoCollection<Document> logs = db.getCollection("logs");</pre>
<pre>// Top Queried Competition logs.aggregate(Arrays.asList(Aggregates.group("\$competition", Accumulators.sum("count", 1)), Aggregates.sort(Sorts.descending("count")), Aggregates.limit(1)))</pre>
<pre>// Most Common Query Type logs.aggregate(Arrays.asList(Aggregates.group("\$type", Accumulators.sum("count", 1)), Aggregates.sort(Sorts.descending("count")), Aggregates.limit(1)))</pre>
<pre>// Recent Logs Recent Logs</pre>
<pre>// Average Results per Type logs.aggregate(Arrays.asList(Aggregates.group("\$type", Accumulators.avg("averageResults", "\$resultCount"))</pre>


```

))
// Pie Chart: Competition → Count
logs.aggregate(Arrays.asList(
    Aggregates.group("$competition", Accumulators.sum("count",
1)),
    Aggregates.sort(Sorts.descending("count"))
))

```

All data is passed into the request scope using `req.setAttribute(...)`

The dashboard JSP renders:

- Most Queried Competition (top header)
- Most Common Type (e.g., top scorers, matches)
- Table of Recent Queries (last 5)
- Table of Average Result Counts per Type
- Google Pie Chart to visualize query distribution

```

<table>
  <tr>

    <th>Competition</th><th>Type</th><th>ID</th><th>Results
</th><th>Timestamp</th>
  </tr>
  <% for (Document doc : recentLogs) { %>
  <tr>
    <td><%= doc.getString("competition") %></td>
    ...
  </tr>
  <% } %>
</table>

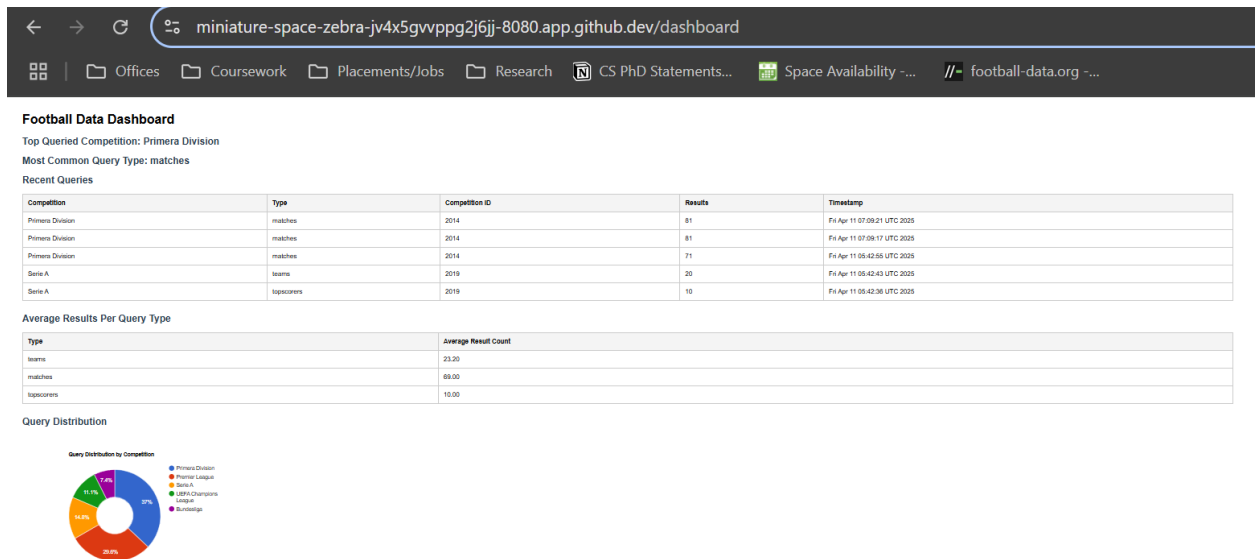
```

This dashboard reveals:

- Which leagues are most viewed?
- What types of data (matches, teams, top scorers) are in demand?
- When was the last query made?
- What's the average size of results returned by API types?

- Trend visualization of engagement by competition via Pie Chart

The Dashboard:



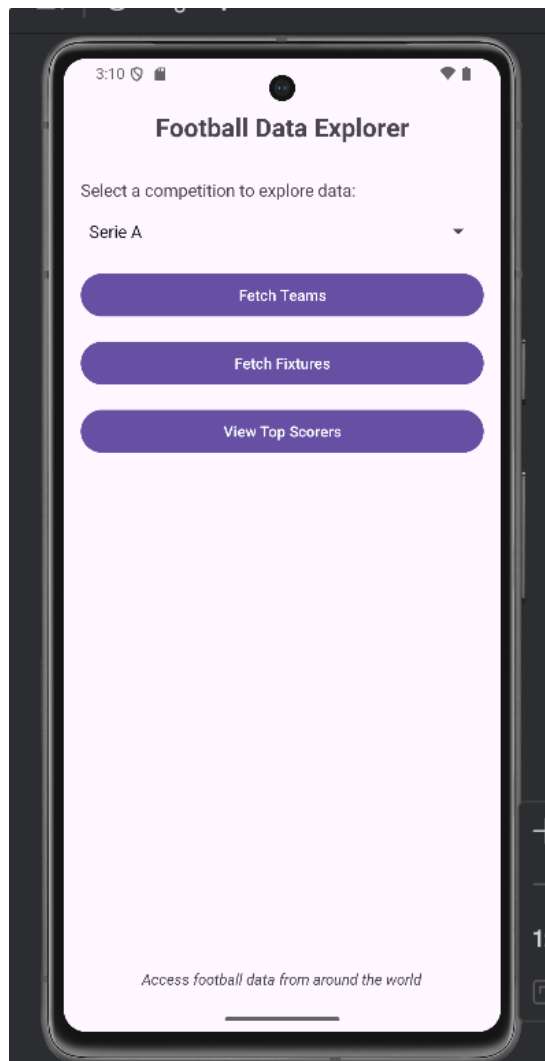
Mongo DB collections:

Here is a sample of the data collected in the Mongo DB

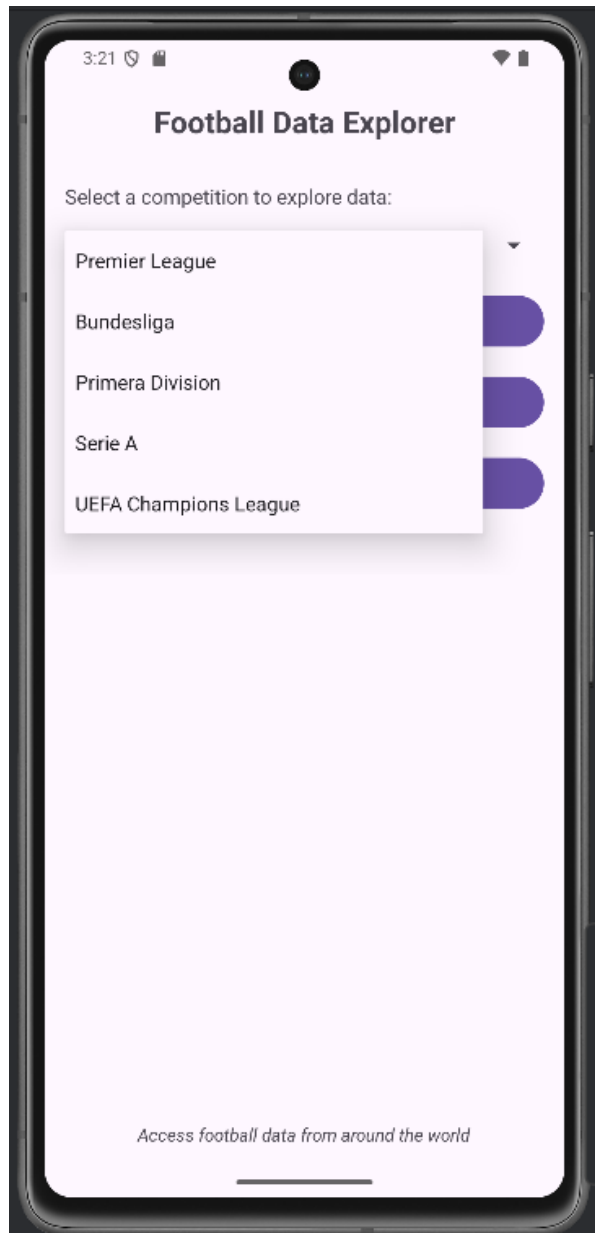
```
_id: ObjectId('67f8ab40b4e16501d2b2e490')
competition: "Primera Division"
type: "topscorers"
competitionId: 2014
resultCount: 10
timestamp: 1744350016356
```

Andriod App UI:

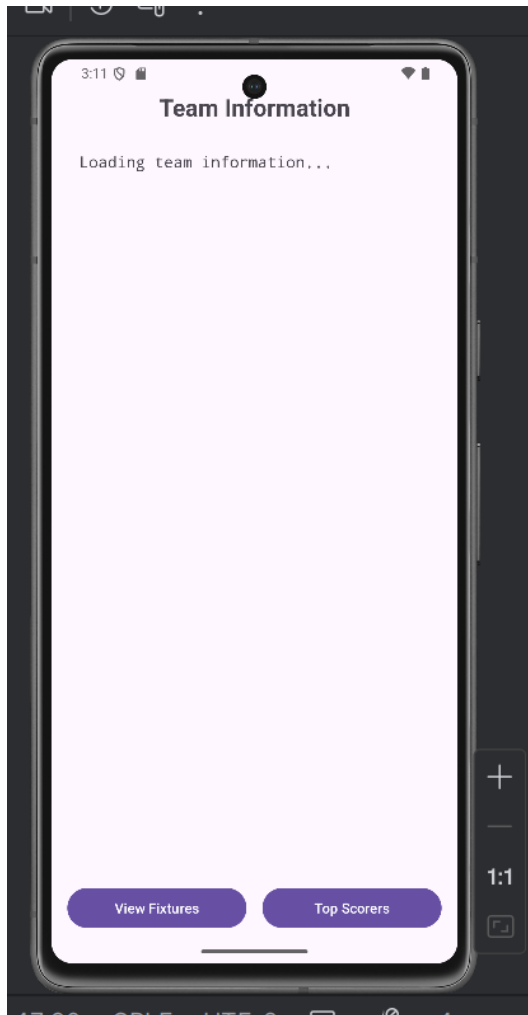
Landing Screen:



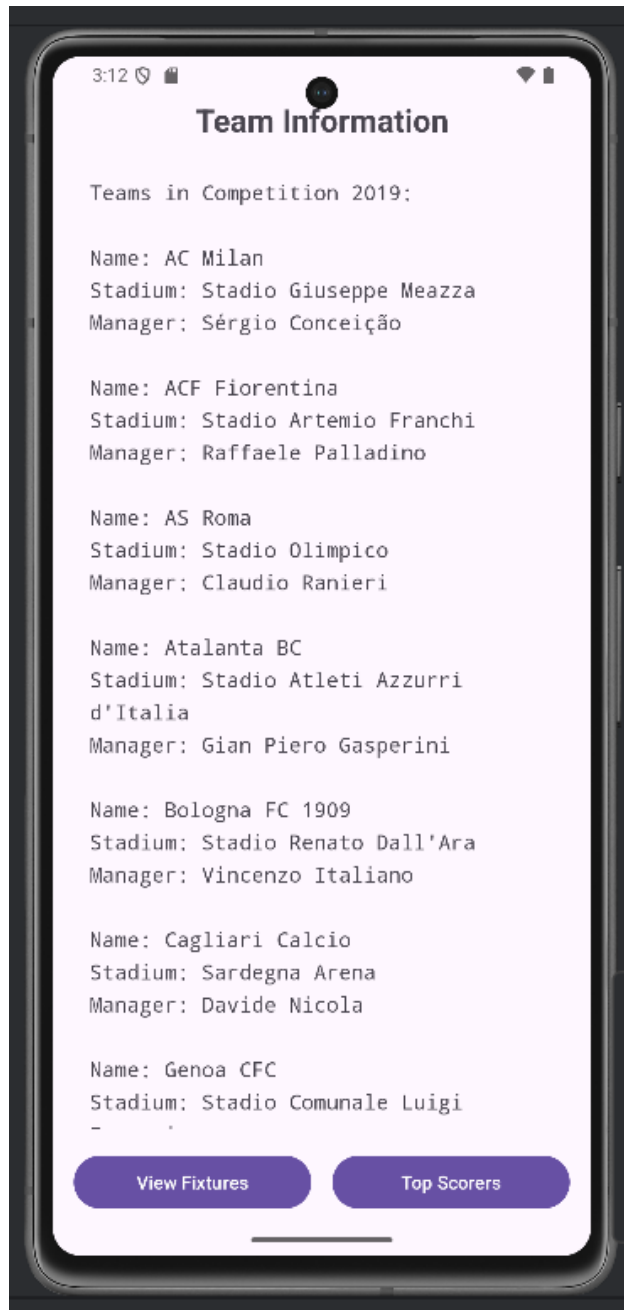
Drop Down menu with all the competitions:



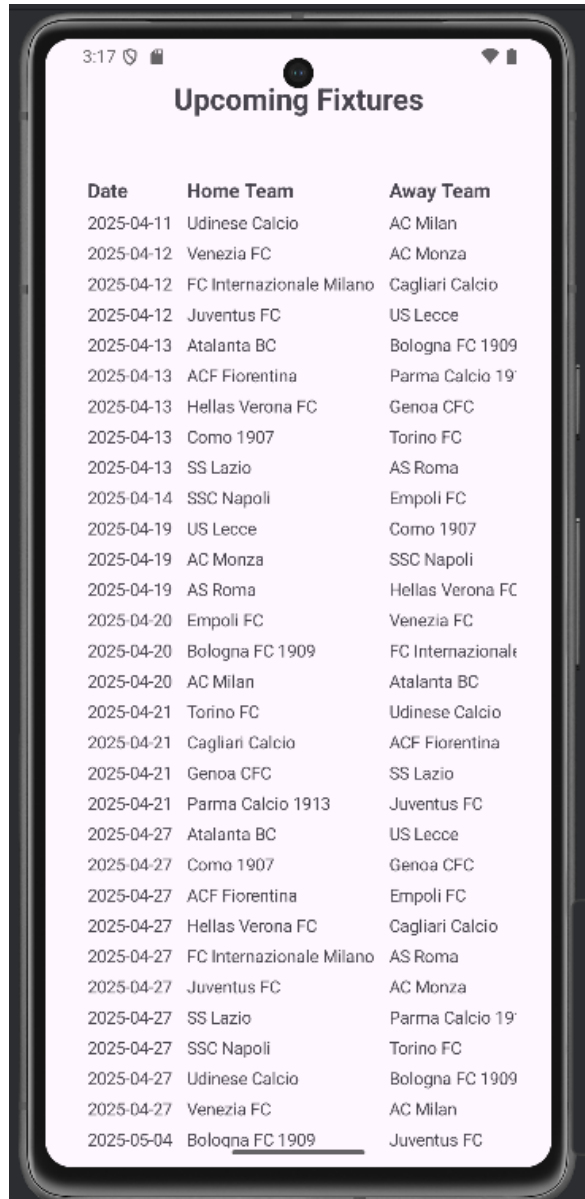
Loading Screen:



Team Information Screen:



Upcoming Fixtures Screen:



Date	Home Team	Away Team
2025-04-11	Udinese Calcio	AC Milan
2025-04-12	Venezia FC	AC Monza
2025-04-12	FC Internazionale Milano	Cagliari Calcio
2025-04-12	Juventus FC	US Lecce
2025-04-13	Atalanta BC	Bologna FC 1909
2025-04-13	ACF Fiorentina	Parma Calcio 19
2025-04-13	Hellas Verona FC	Genoa CFC
2025-04-13	Como 1907	Torino FC
2025-04-13	SS Lazio	AS Roma
2025-04-14	SSC Napoli	Empoli FC
2025-04-19	US Lecce	Como 1907
2025-04-19	AC Monza	SSC Napoli
2025-04-19	AS Roma	Hellas Verona FC
2025-04-20	Empoli FC	Venezia FC
2025-04-20	Bologna FC 1909	FC Internazionale
2025-04-20	AC Milan	Atalanta BC
2025-04-21	Torino FC	Udinese Calcio
2025-04-21	Cagliari Calcio	ACF Fiorentina
2025-04-21	Genoa CFC	SS Lazio
2025-04-21	Parma Calcio 1913	Juventus FC
2025-04-27	Atalanta BC	US Lecce
2025-04-27	Como 1907	Genoa CFC
2025-04-27	ACF Fiorentina	Empoli FC
2025-04-27	Hellas Verona FC	Cagliari Calcio
2025-04-27	FC Internazionale Milano	AS Roma
2025-04-27	Juventus FC	AC Monza
2025-04-27	SS Lazio	Parma Calcio 19
2025-04-27	SSC Napoli	Torino FC
2025-04-27	Udinese Calcio	Bologna FC 1909
2025-04-27	Venezia FC	AC Milan
2025-05-04	Bologna FC 1909	Juventus FC

Top Scorers Screen:

Name	Matches	Goals
Mateo Retegui	29	22
Moise Kean	28	17
Marcus Thuram-Ulien	30	14
Ademola Lookman	26	13
Riccardo Orsolini	25	11
Romelu Lukaku	29	11
Artem Dovbyk	29	11
Lautaro Martinez	28	11
Lorenzo Lucca	30	10
Nikola Krstović	30	10